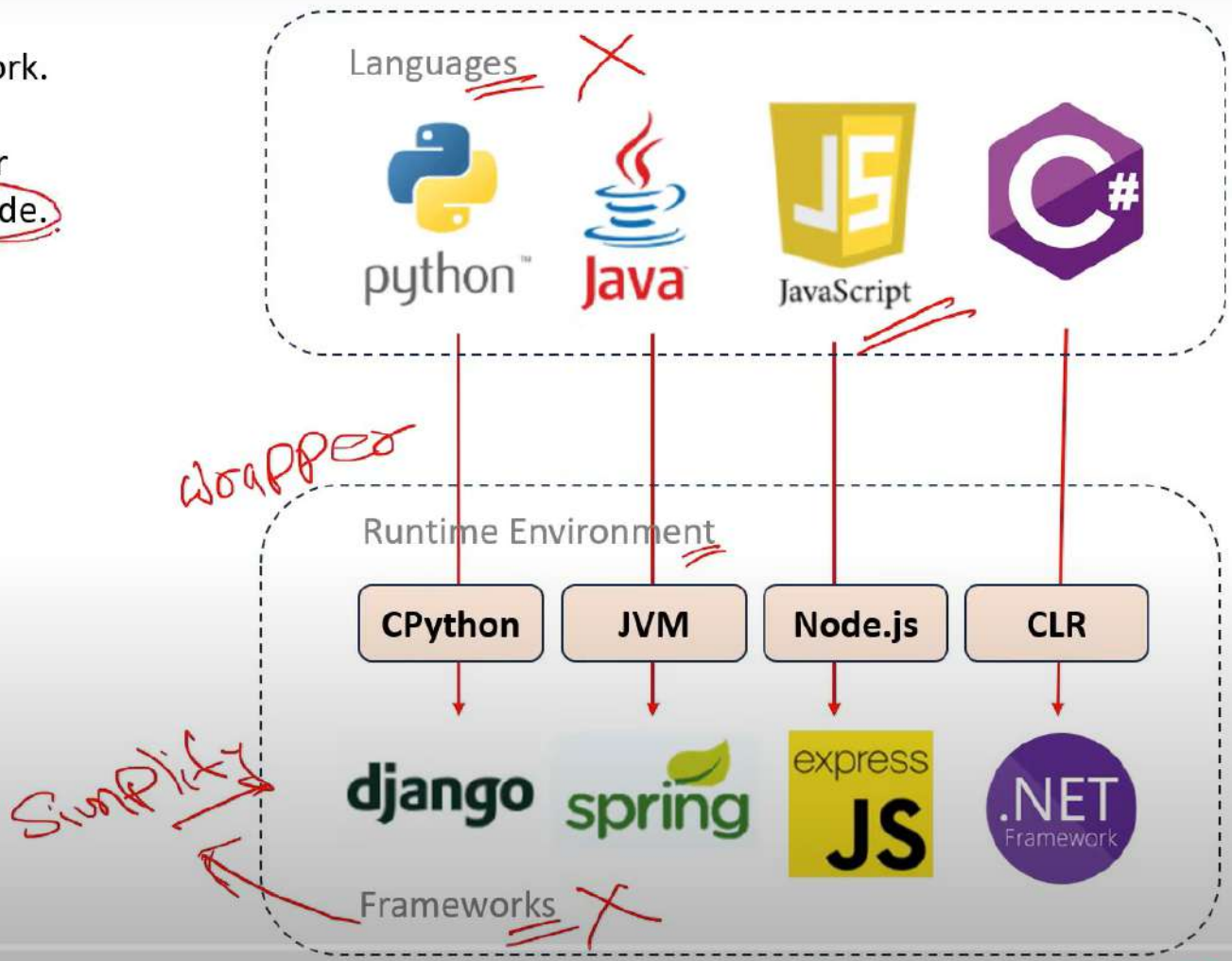


Node.js - Top 100 Interview Questions and Answers
Q. What is Node.js? V. IMP.

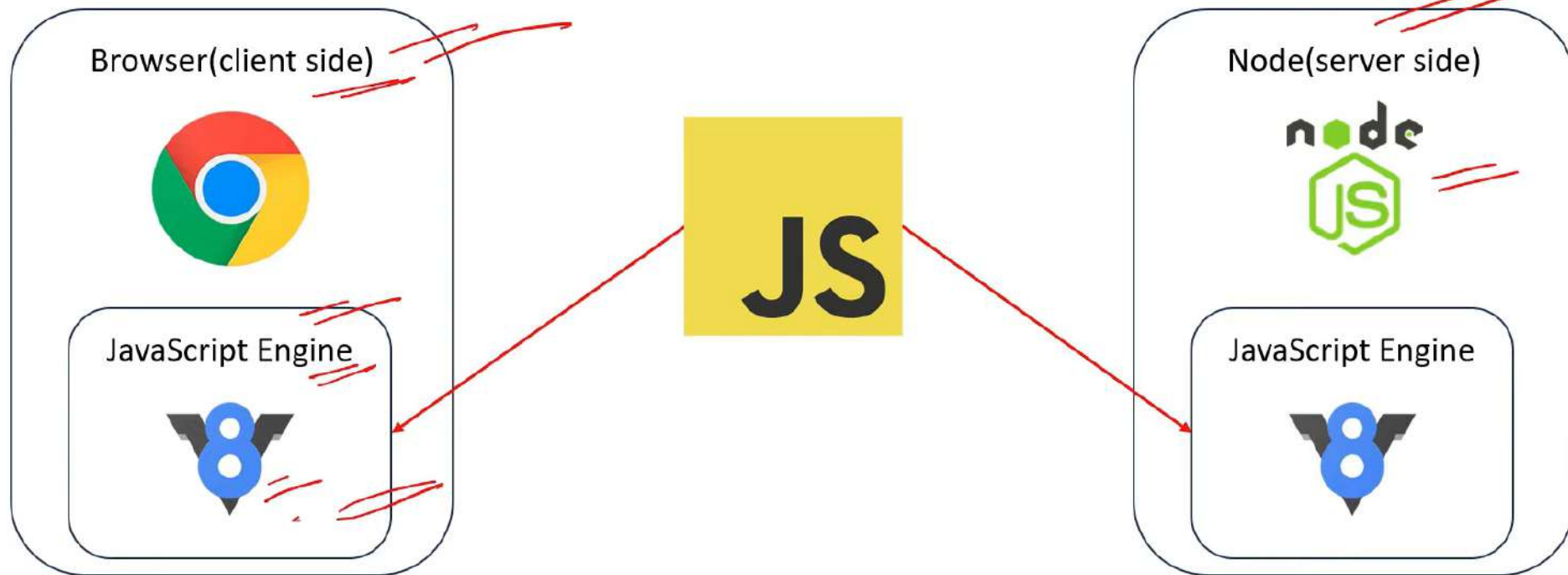
- ❖ Node is neither a language nor a framework.
- ❖ Node/ Node.js a runtime environment for executing JavaScript code on the server side.



Q. How Node is a runtime environment on server side? What is V8?



- ❖ Browsers execute JavaScript on the client side, and similarly, Node.js executes JavaScript on the server side.
- ❖ V8 is a JavaScript engine for the JavaScript language.





Chapters

FUNDAMENTALS

1. Node.js Basics
2. Main Features of Node.js
3. Project setup & Modules
4. Top built in modules

EXPRESS

5. Express Basics
6. Middleware
7. Types of Middleware's
8. Routing - I
9. Routing - II
10. Template Engines

REST API

11. REST API Basics
12. HTTP Methods & Status Codes
13. CORS, Serialization, Deserialization, Others
14. Authentication & Authorization

NODE - OTHERS

- 15 Error Handling & Debugging
16. Security, Performance & Deployment
17. Testing & Web Socket

MONGO-DB

18. MongoDB Basics
19. Setup & CRUD operations
20. Query Operators & Projection
21. Indexes
22. Mongoose



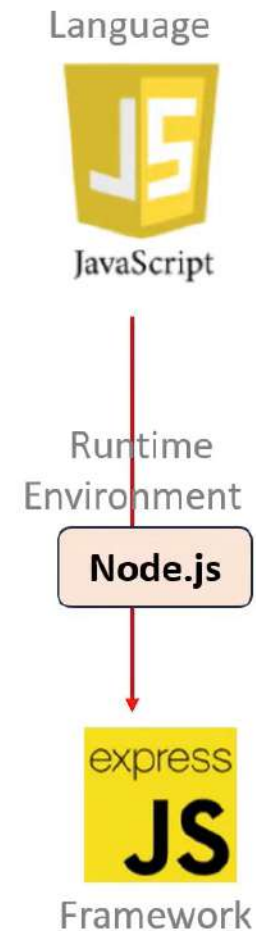
- 23 Application based questions.
24. Scenario based short questions
25. JavaScript Essentials for Node



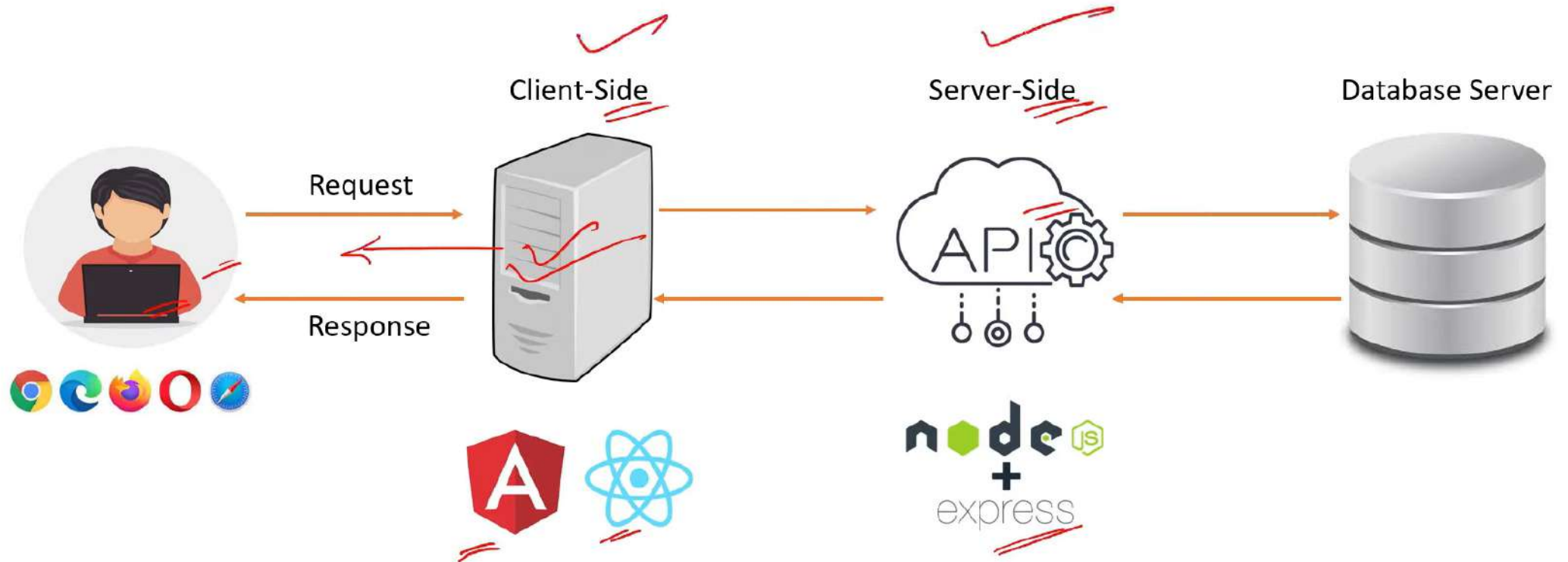
Q. What is the difference between Node.js & Express.js?



- ❖ Node.js is a **runtime environment** that allows the execution of JavaScript code server-side.
- ❖ Express.js is a **framework** built on the top Node.js.
 - ❖ It is designed to simplify the process of building web applications and APIs by providing a set of features like simple routing system, middleware support etc.



Q. What are the differences between Client-Side(Browser) & Server-Side(Node.js)?



Q. What are the differences between **Client Side(Browser)** & **Server-Side(Node.js)**?



	Client-Side (Browser)	Server-Side (Node.js/Server)
Environment Location	Runs on the user's web browser.	Runs on the server.
Primary Languages	HTML, CSS, JavaScript.	JavaScript
<u>Document/ Window/ Navigator/ Event Objects</u>	Yes <u>document, window, navigator, event</u>	No
<u>Request/ Response/ Server/ Database Object</u>	No	Yes
Responsibilities	Handles UI <u>display, interactions,</u> and client-side logic.	Handles <u>business logic, data storage/ access, authentication, authorization</u> etc.



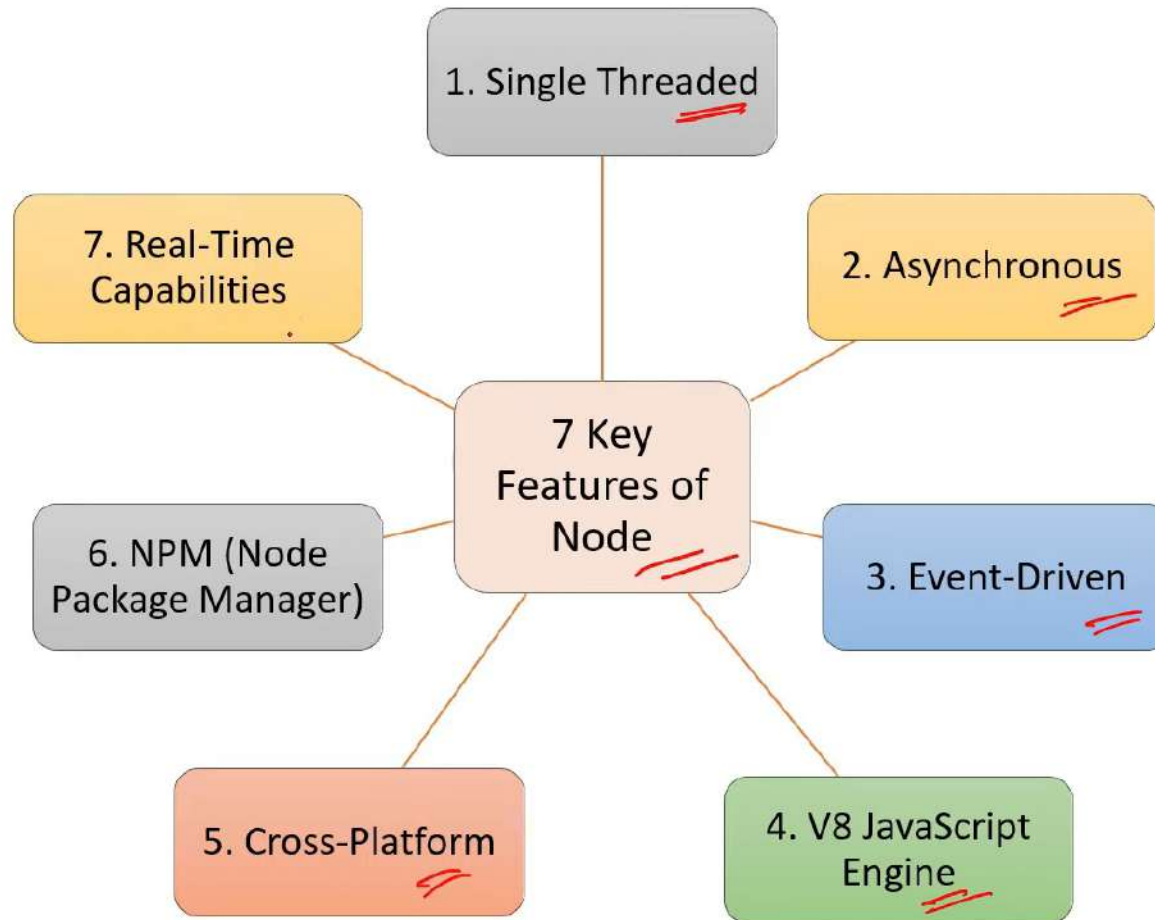
2. Main Features of Node.js



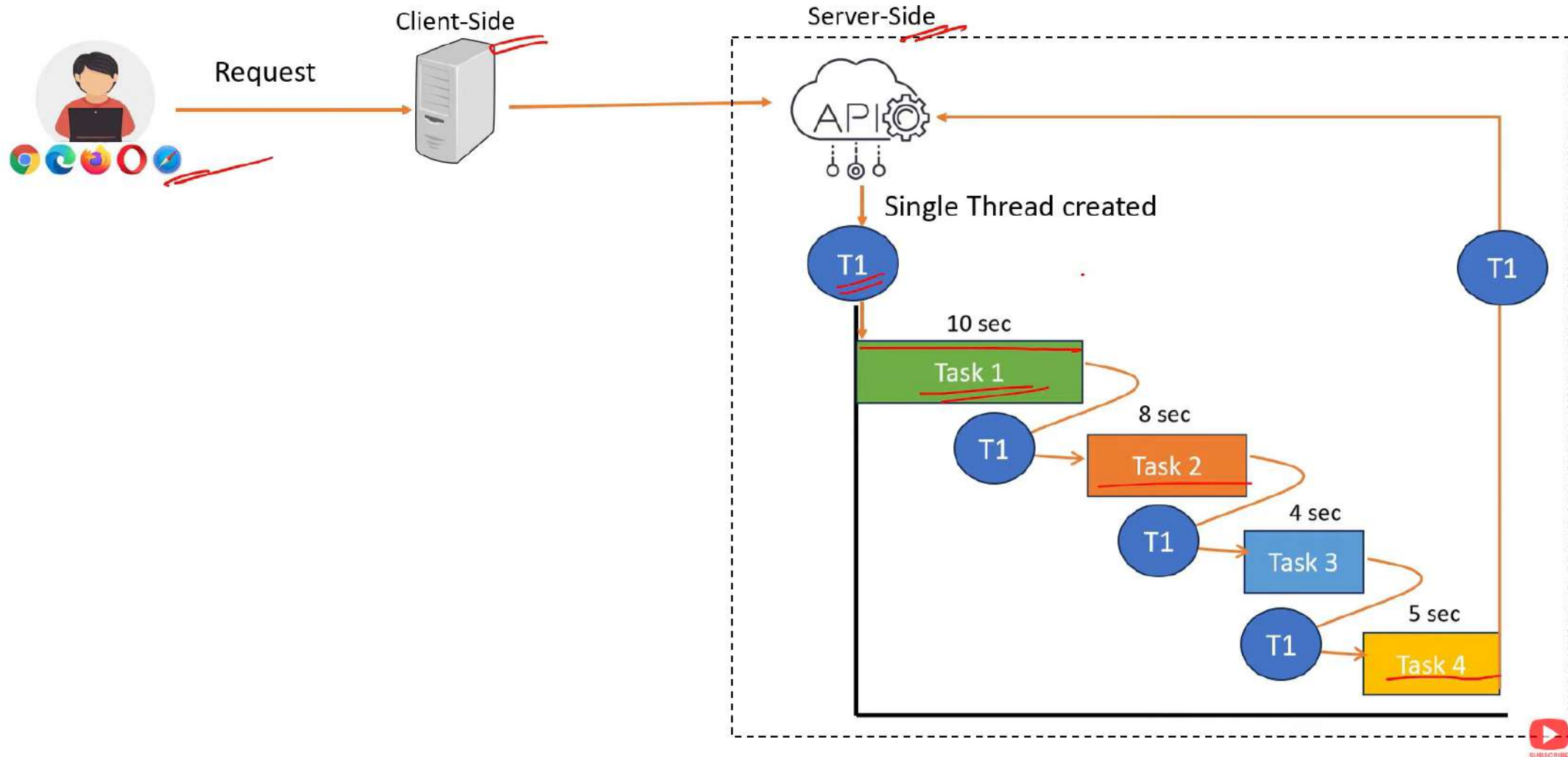
- Q. What are the **7 Main Features** of Node.js?
- Q. What is **Single Threaded** Programming?
- Q. What is **Synchronous** Programming?
- Q. What is **Multi Threaded** Programming?
- Q. What is **Asynchronous** Programming?
- Q. What is the difference between **Synchronous** & **Asynchronous** programming?
- Q. What are **Events**, **Event Emitter**, **Event Queue**, **Event Loop** & **Event Driven**?
- Q. What are the main features & **advantages** of Node.js?
- Q. What are the **disadvantages** of node?



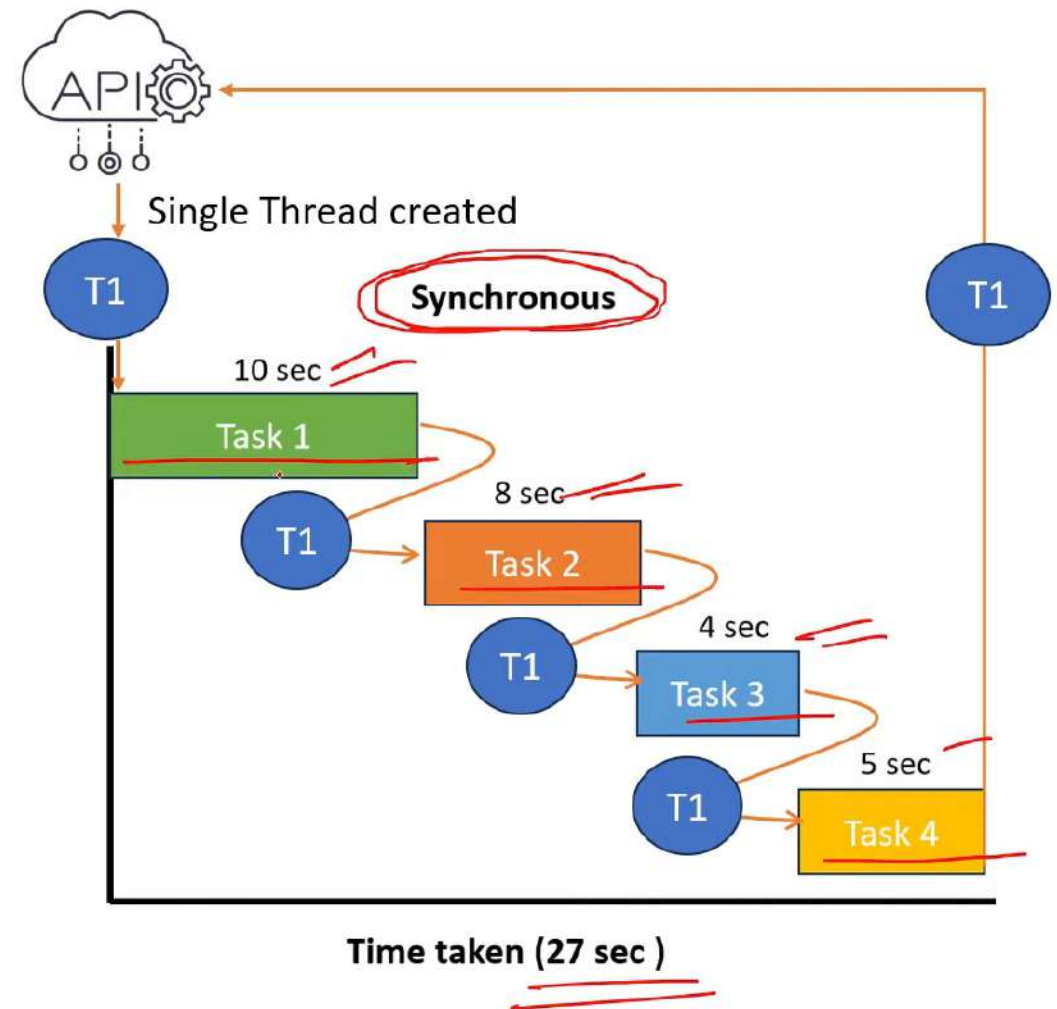
Q. What are the 7 Main Features of Node.js? **V. IMP.**



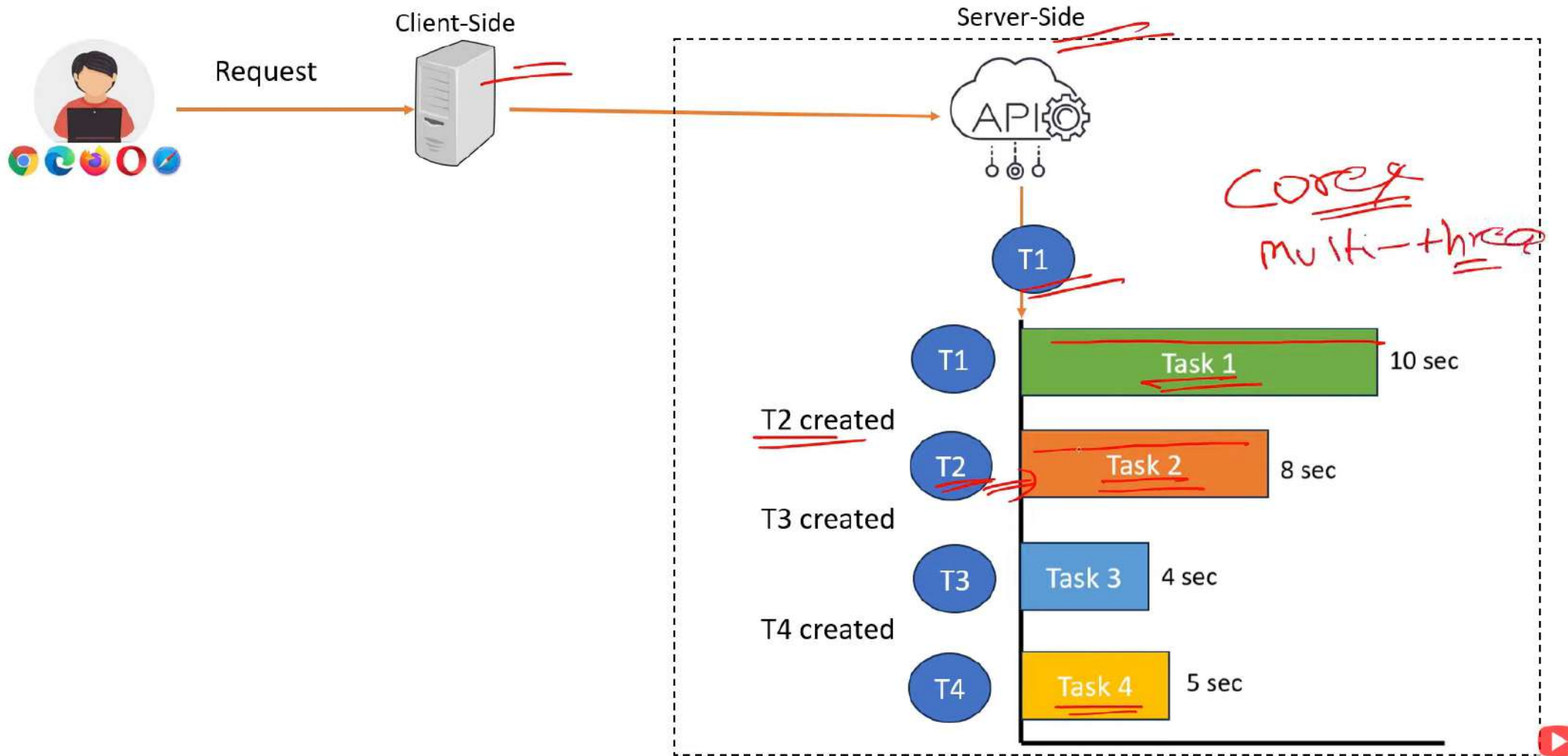
Q. What is Single Threaded Programming?



Q. What is **Synchronous** Programming?



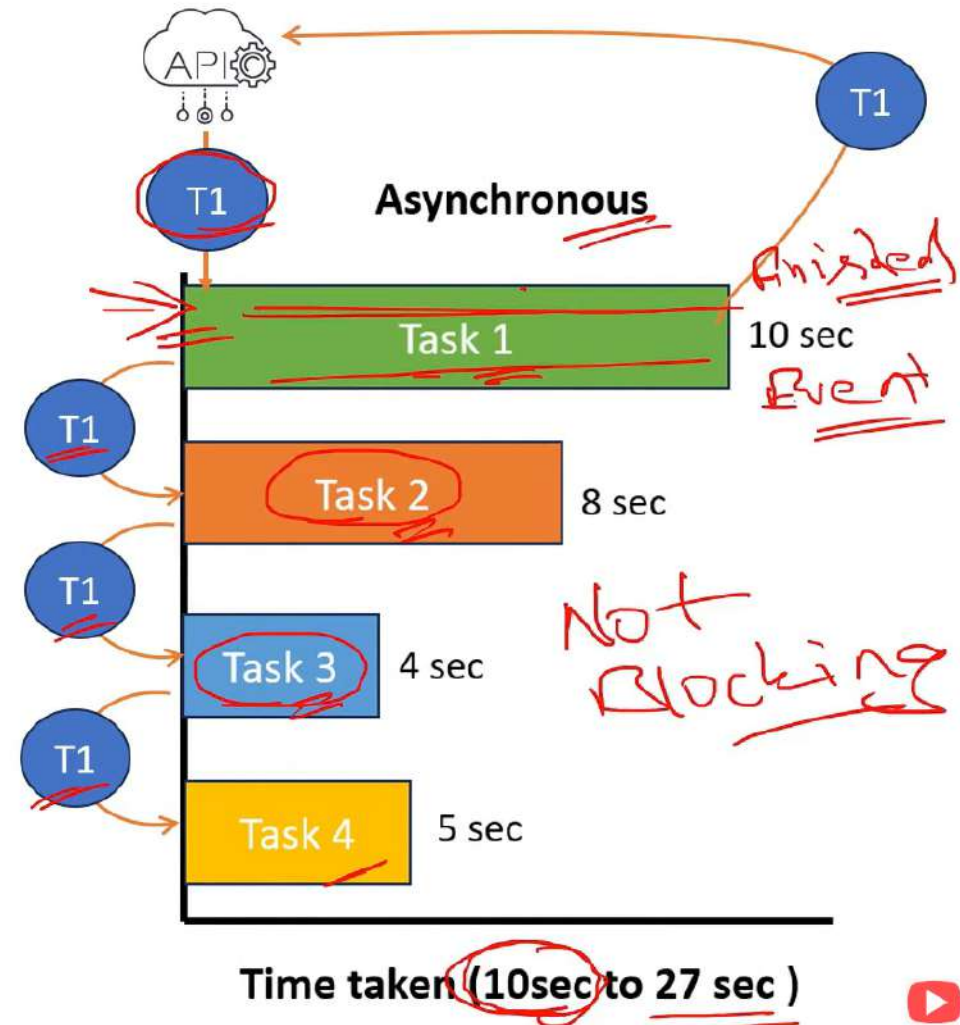
Q. What is Multi Threaded Programming?



Q. What is Asynchronous Programming? **V. IMP.**



- ❖ In Node.js, asynchronous flow can be achieved by its single-threaded, non-blocking, and event-driven architecture.
- ❖ In Node.js, if there are 4 tasks(Task1, Task2, Task3, Task4) to be completed for an event. Then below steps will be executed:
 1. First, Thread T1 will be created.
 2. Thread T1 initiates Task1, but it won't wait for Task1 to complete. Instead, T1 proceeds to initiate Task2, then Task3 and Task4(This asynchronous execution allows T1 to efficiently handle multiple tasks concurrently).
 3. Whenever Task1 completes, an event is emitted.
 4. Thread T1, being event-driven, promptly responds to this event, interrupting its current task and delivering the result of Task1.



Q. What is the difference between Synchronous & Asynchronous programming?

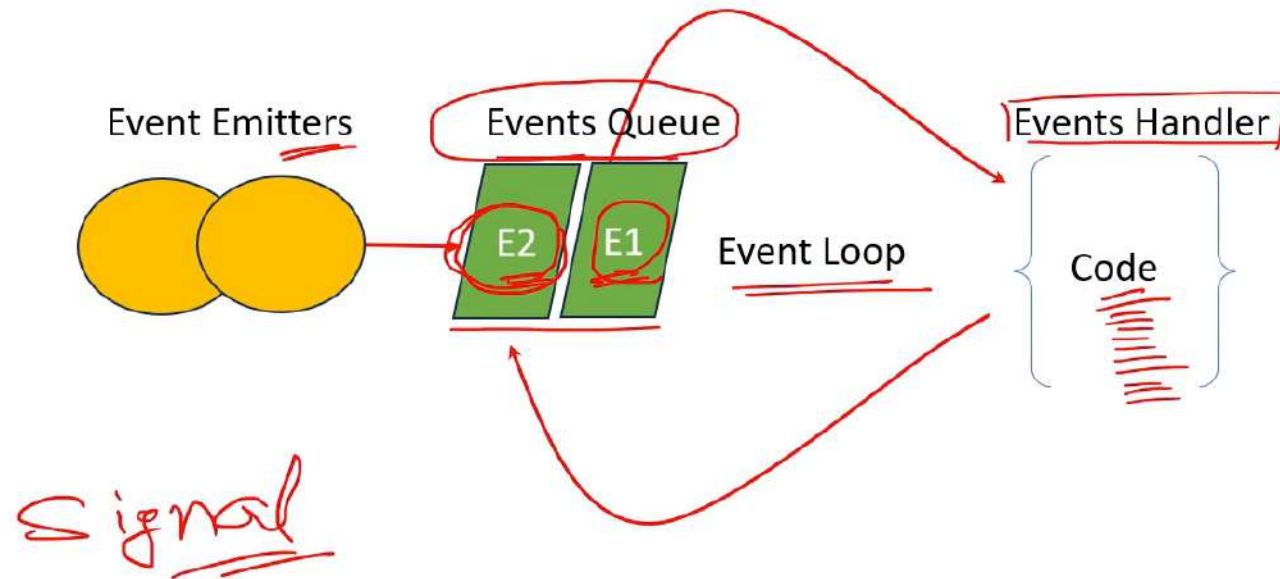


Synchronous programming	Asynchronous programming
1. In synchronous programming, tasks are executed one after another in a sequential manner .	In asynchronous programming, tasks can start, run, and complete in parallel
2. Each task must complete before the program moves on to the next task.	Tasks can be executed independently of each other. 
3. Execution of code is blocked until a task is finished.	Asynchronous operations are typically non-blocking. 
4. Synchronous operations can lead to blocking and unresponsiveness.	It enables better concurrency and responsiveness. 

Q. What are Events, Event Emitter, Event Queue, Event Loop & Event Driven?



- ❖ **Event:** Signals that something has happened in a program.
- ❖ **Event Emitter:** Create or emit events.
- ❖ **Event Queue:** Events emitted queued(stored) in event queue.
- ❖ **Event Handler(Event Listener):** Function that responds to specific events
- ❖ **Event Loop:** The event loop picks up event from the event queue and executes them in the order they were added.
- ❖ **Event Driven Architecture:** It means operations in Node are drive or based by events.



Q. What are the main features & advantages of Node.js?



Features



Advantages



1. Asynchronous

Enables handling multiple concurrent requests & non blocking execution of thread.

2. V8 JS Engine

Built on the V8 JS engine from Google Chrome, Node.js executes code fast.

3. Event-Driven Architecture

Efficient handling events Great for real time applications like chat applications, gaming applications (using web sockets) where bidirectional communication is required.

4. Cross-Platform

Supports deployment on various operating systems, enhancing flexibility.

5. JavaScript

Coding in JS language therefore no need to learn a new language.

Suitable for building scalable applications that can handle increased loads.



Q. What are the disadvantages of node? When to use and when not to use Node?



When to Use Node.js?

- ❖ Ideal for **real-time applications** like chat applications, online gaming, and collaborative tools due to its event-driven architecture.
- ❖ Excellent for building **lightweight and scalable RESTful APIs** that handle a large number of concurrent connections.
- ❖ Well-suited for building **microservices-based architectures**, enabling modular and scalable systems.

When not to use Node.js(disadvantages):

- ❖ **CPU-Intensive Tasks:** Avoid for applications that involve heavy CPU processing(Image/Video Processing, Data Encryption/Decryption,) as Node.js may not provide optimal performance in such scenarios because it is single threaded and for heavy computation multi-threaded is better.

3. Project Setup & Modules



- Q. How to **setup** node.js project?
- Q. What is **NPM**? What is the role of **node_modules** folder?
- Q. What is the role of **package.json** file in Node?
- Q. What are **Modules** in Node?
- Q. How many ways are there to **Export** a module?
- Q. What will happen if you don't export the module?
- Q. How to **import** single and multiple functions from a module?
- Q. What is **module wrapper function**?
- Q. What are the **Types of modules** in Node?



Q. How to setup node.js project?



❖ 7 Steps for setting up node.js project in VS Code:

1. Download Node.js installer & install it.
2. Download VSCode editor installer & install it.
3. Create a new folder (myFirstProject) anywhere.
4. Open VSCode and open myFirstProject folder from it.
5. Open terminal in VSCode and run this command: npm init -y.
6. Create a new JS file(app.js) within your project folder(myFirstProject).
7. Run your Node.js application by using this command in the terminal - node app.js.



Q. How to **setup** node.js project?



The screenshot shows the Visual Studio Code interface with a dark theme. The Explorer sidebar on the left shows a project named 'NODE-C...' with a subfolder 'node_modules' and files 'app.js', 'package-lock.json', and 'package.json'. The main editor area has a tab for 'JS app.js' with a red 'X' icon. The code in the editor is as follows:

```
1 // app.js
2 const message = "Interview Happy";
3
4 console.log(message);
5
6 // Output: Interview Happy
7
8
```

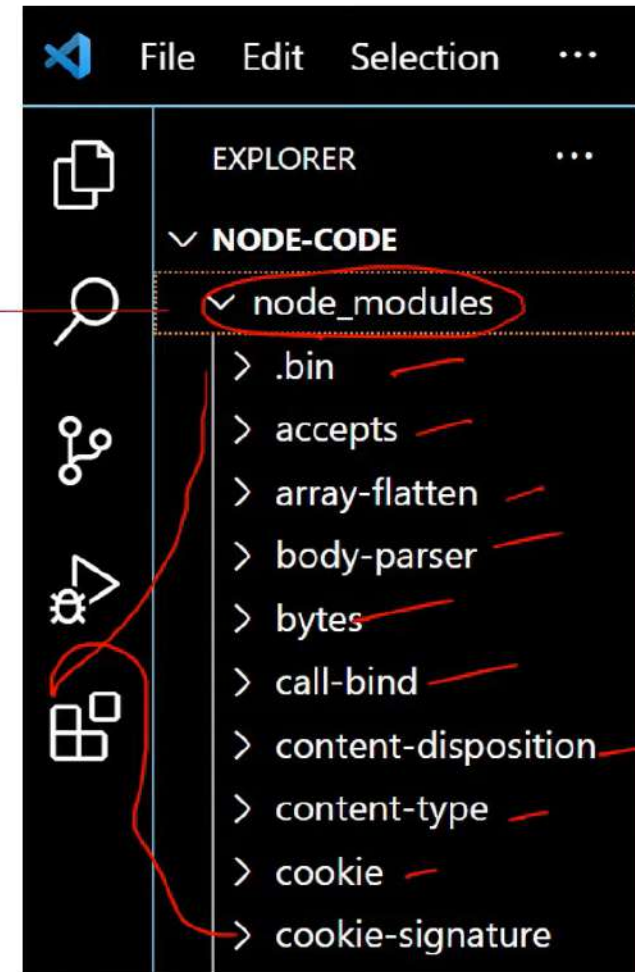
At the bottom, the TERMINAL panel is active, showing a PowerShell prompt. The command 'node app.js' has been executed, and the output 'Interview Happy' is displayed. The command and its output are highlighted with red boxes.



Q. What is NPM? What is the role of node_modules folder?

❖ NPM(Node Package Manager) is used to manage the dependencies for your Node project.

❖ node_modules folder contains all the dependencies of the node project.



Q. What is the role of package.json file in Node?



- ❖ The package.json file contains project metadata (information about the project). For example, project name, version, description, author, license etc.

```
EXPLORER  ...  {} package.json X
✓ NODE-C...  {} package.json > ...
  > node_modules
  JS app.js
  {} package-lock.json
  {} package.json

1  {
2    "name": "node-code",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \\\"Error:
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "dependencies": {
13     "express": "^4.18.2"
14 }
15 }
```



Q. What are Modules in Node? What is the difference between a function & module?



- ❖ A module contains a specific functionality that can be easily reused within a Node.js application.
- ❖ Ideally in node.js, a JavaScript file can be treated as a module.

```
// myModule1.js
function sayHello(name) {
  console.log("Hello, " + name);
}

module.exports = sayHello;
.....
```

- ❖ A module is a broader concept that encapsulates functionality, while a function is a specific set of instructions within that module.
- ❖ Modules can contain multiple functions and variables.

Modules

EXPLORER

✓ NODE-C... [Icons]
> .vscode
> node_modules
JS app-use.js
JS app.js
JS event-argument.js
JS event-example.js
JS fs-example.js



Q. How many ways are there to Export a module?



```
// Exporting a function
function sayHello(name) {
  console.log("Hello, " + name);
}

// Exporting a variable
const pi = 3.14159;

// Exporting an object
const myObject = {
  key: "value",
};

// Using module.exports
module.exports.sayHello = sayHello;
module.exports.pi = pi;
module.exports.myObject = myObject;
```



```
// Exporting a function
exports.sayHello = function (name) {
  console.log("Hello, " + name);
};

// Exporting a variable
exports.pi = 3.14159;

// Exporting an object
exports.myObject = {
  key: "value",
};
```



Q. How to **import** single and multiple functions from a module?



```
// module2.js

function sayHello(name) {
| console.log("Hello, " + name + "!");
}

// To export single function
module.exports = sayHello;
.....

// To export multiple function
// module.exports.sayHello = sayHello
// module.exports.sayGoodbye = sayGoodbye
```

```
// app.js

// Importing the module2 module
const module2 = require('./module2');

// To import single function
module2('Happy'); sayHello

// Output: Hello, Happy!
```



SUBSCRIBE



```
// app.js
const message = 'Interview Happy';

console.log(message);

// OutputL Interview Happy
```

```
// Wrapper function generated by Node

(function (exports, require, module,
  __filename, __dirname) {
  const message = "Interview Happy";

  console.log(message);

  // Output: Interview Happy
});
```

Q. What are the Types of modules in Node?



1. Built-in Module
(Core Modules)



2. Local Modules



3. Third-Party Modules

install
3



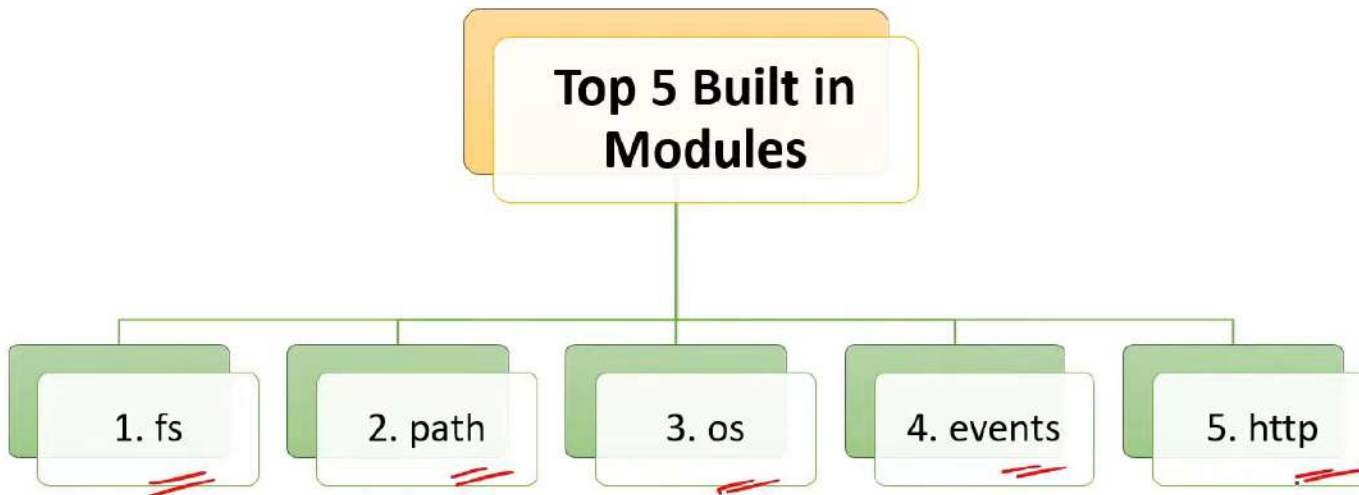
4. Top built in modules



- Q. What are the **Top 5 built in modules** commonly used in node projects?
- Q. Explain the role of **fs module**? Name some functions of it?
- Q. Explain the role of **path module**? Name some functions of it?
- Q. Explain the role of **OS module**? Name some functions of it?
- Q. Explain the role of **events module**? How to handle events in Node?
- Q. What are **Event Arguments**?
- Q. What is the difference between a **function** and an **event**?
- Q. What is the role of **http module** in node?
- Q. What is the role **createServer() method** of http module?



Q. What are the Top 5 built in modules commonly used in node projects?





Q. Explain the role of fs module? Name some functions of it?

- ❖ fs (File System) module in Node provides a set of methods for **interacting with the file system.**

```
// fs-example.js
const fs = require("fs");

// Reading the contents of a file asynchronously
fs.readFile("fs.txt", "utf8", (err, data) => {
  if (err) {
    return;
  }
  console.log("File contents:", data);
});

// Writing to a file asynchronously
const contentToWrite = "Some content";
fs.writeFile("fs.txt", contentToWrite, "utf8", (err) => {
  if (err) {
    return;
  }
  console.log("Write operation complete.");
});
```



Q. Explain the role of **fs module**? Name some functions of it?



❖ **7 Main functions of fs module:**

1. **fs.readFile()**

Reads the content of the file specified

2. **fs.writeFile()**

Writes data to the specified file, creating the file if it does not exist.

3. **fs.appendFile()**

Appends data to a file. If the file does not exist, it is created.

4. **fs.unlink()**

Deletes the specified file.

5. **fs.readdir()**

Reads the contents of a directory.

6. **fs.mkdir()**

Creates a new directory.

7. **fs.rmdir()**

Removes the specified directory.





Q. Explain the role of path module? Name some functions of it?

- ❖ Path module provides utilities for joining, resolving, parsing, formatting, normalizing, and manipulating paths.

```
// path-example.js

const path = require('path');

// Joining Path Segments
const fullPath = path.join('/docs', 'file.txt');
console.log(fullPath);
// Output: /docs/file.txt

// Parsing Path
const parsedPath = path.parse('/docs/file.txt');
console.log(parsedPath);
/*
Output: {root: '/', dir: '/docs', base: 'file.txt',
ext: '.txt', name: 'file'
}
*/
```





Q. Explain the role of **path module**? Name some functions of it?

❖ 5 Main functions of path module:

```
const path = require('path');  
  
// Joining path segments together  
const fullPath = path.join(__dirname, 'folder', 'file.txt');  
  
// Resolving the absolute path  
const absolutePath = path.resolve('folder', 'file.txt');  
  
// Getting the directory name of a path  
const directoryName = path.dirname('/folder/file.txt');  
  
// Getting the file extension of a path  
const fileExtension = path.extname('/folder/file.txt');  
  
// Parsing a path into an object with its components  
const pathObject = path.parse('/folder/file.txt');
```





Q. Explain the role of OS module? Name some functions of it?

- ❖ The os module in Node.js provides a set of methods for interacting with the operating system.
- ❖ Operation system can be used by developers for building cross-platform applications or performing system-related tasks in Node.js applications.

```
const os = require('os');  
  
// 1. Get Platform Information  
console.log(os.type());  
// Output: 'Windows_NT' or 'Linux'...  
  
// 2. Get Current User Information  
console.log(os.userInfo());  
// Output: {uid: -1, gid: -1, username: 'anaya'...}  
  
// 3. Get Memory Information in bytes  
console.log(os.totalmem()); // Output: 14877265920  
console.log(os.freemem()); // Output: 4961570816
```

Handwritten diagram showing 'Windows' and 'Linux' with arrows pointing to 'Cross'.

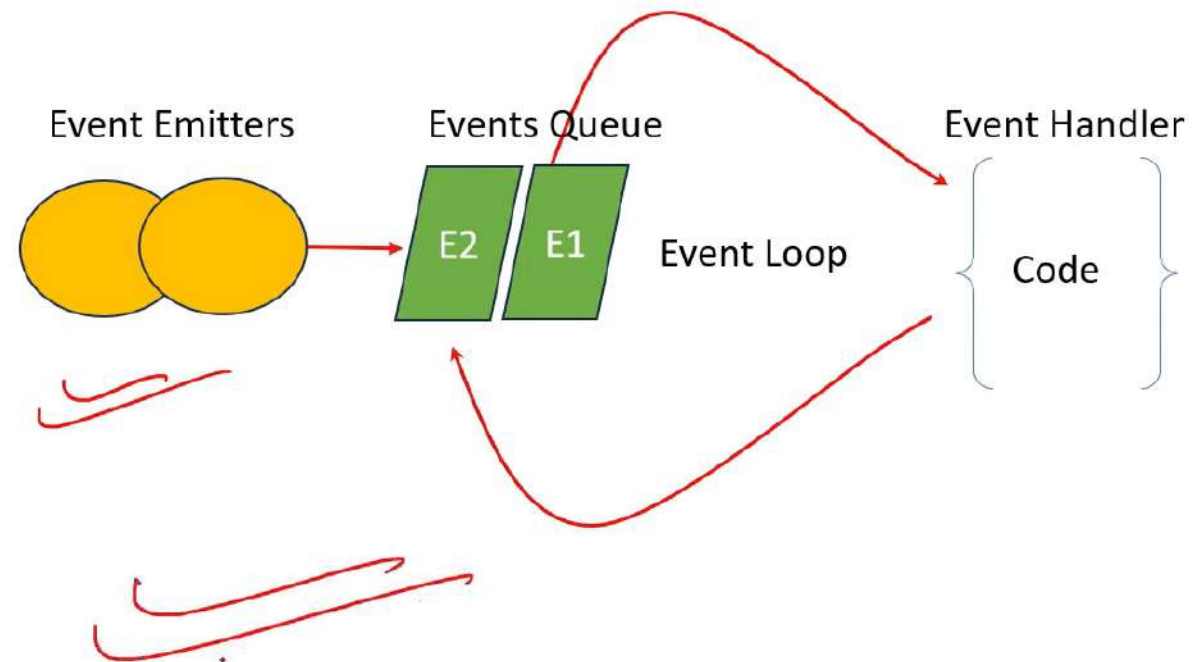
```
graph TD
    Windows[Windows] --> Cross[Cross]
    Linux[Linux] --> Cross
```



Q. Explain the role of events module? How to handle events in Node?



- ❖ **Event:** Signals that something has happened in a program.
- ❖ **Event Emitter:** Create or emit events.
- ❖ **Event Queue:** Events emitted queued(stored) in event queue.
- ❖ **Event Handler(Event Listener):** Function that responds to specific events
- ❖ **Event Loop:** The event loop picks up event from the event queue and executes them in the order they were added.
- ❖ **Event Driven Architecture:** It means operations in Node are drive or based by events.



Q. Explain the role of **events** module? How to **handle events** in Node?



```
// import events module
const EventEmitter = require('events');

// Create an instance of EventEmitter class
const myEmitter = new EventEmitter();

// Register an event listener(eventName)
myEmitter.on('eventName', () => {
| console.log('Event occurred');
| });

// Emit the event
myEmitter.emit('eventName');

// Output: Event occurred
```



SUBSCRIBE



Q. Explain the role of **events** module? How to **handle events** in Node?

1. **events module** is used to handle events.
2. **EventEmitter** class of events module is used to register event listeners and emit events.
3. An **event listener** is a function that will be executed when a particular event occurs.
4. **on()** method is used to register event listeners.

```
// import events module
const EventEmitter = require('events');

// Create an instance of EventEmitter class
const myEmitter = new EventEmitter();

// Register an event listener(eventName)
myEmitter.on('eventName', () => {
  console.log('Event occurred');
});

// Emit the event
myEmitter.emit('eventName');

// Output: Event occurred
```



Q. What are Event Arguments?



```
const EventEmitter = require("events");  
  
// Create an instance of EventEmitter class  
const myEmitter = new EventEmitter();  
  
// Register an event listener for the 'eventName' event  
myEmitter.on("eventName", (arg1, arg2) => {  
| console.log("Event occurred with arguments:", arg1, arg2);  
});  
  
// Emit the 'eventName' event with arguments  
myEmitter.emit("eventName", "Arg 1", "Arg 2");  
  
// Output: Event occurred with arguments: Arg 1 Arg 2
```

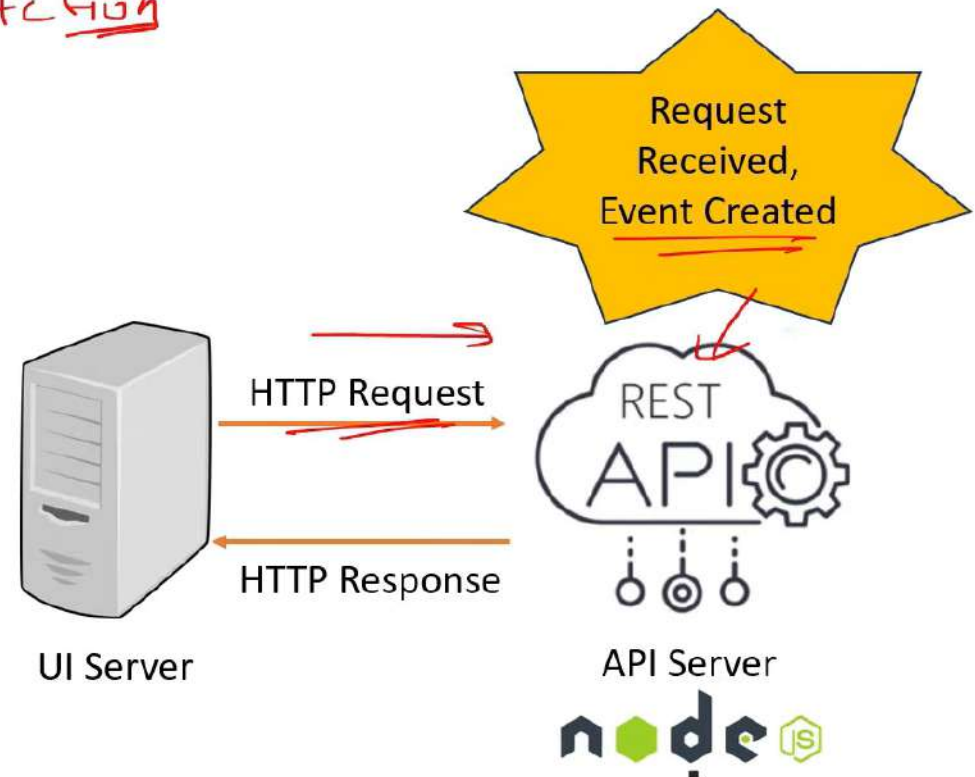


Q. What is the difference between a function and an event?

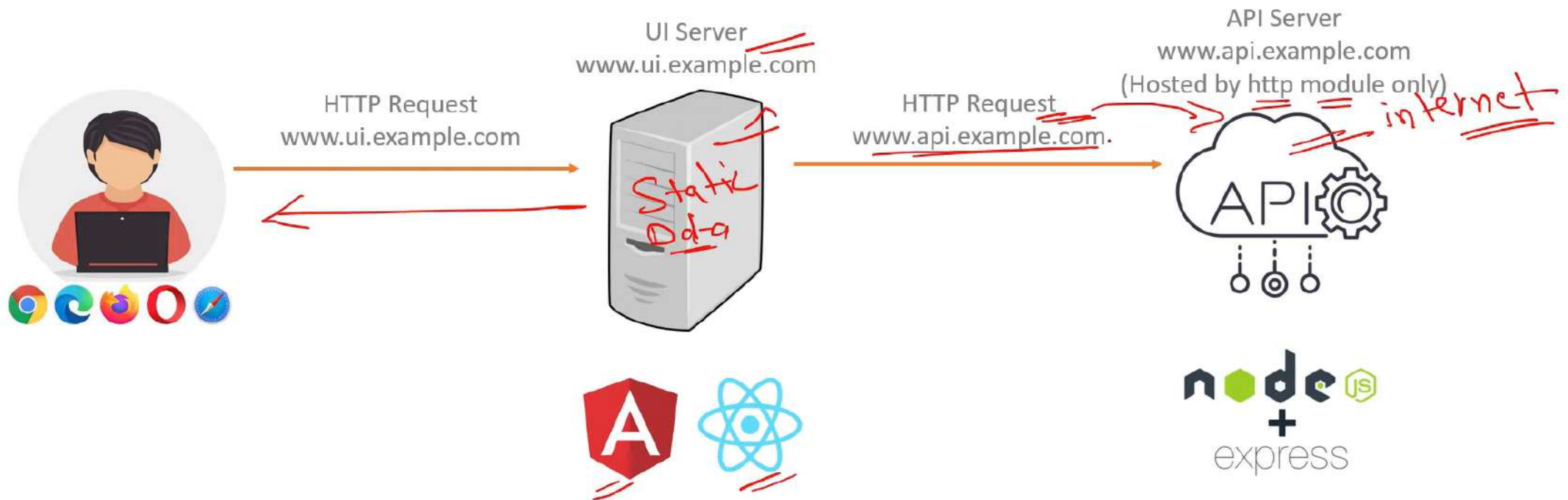


- ❖ A function is a reusable piece of code that performs a specific task when invoked or called.
- ❖ Events represent actions that can be observed and responded to. Events will call functions internally.

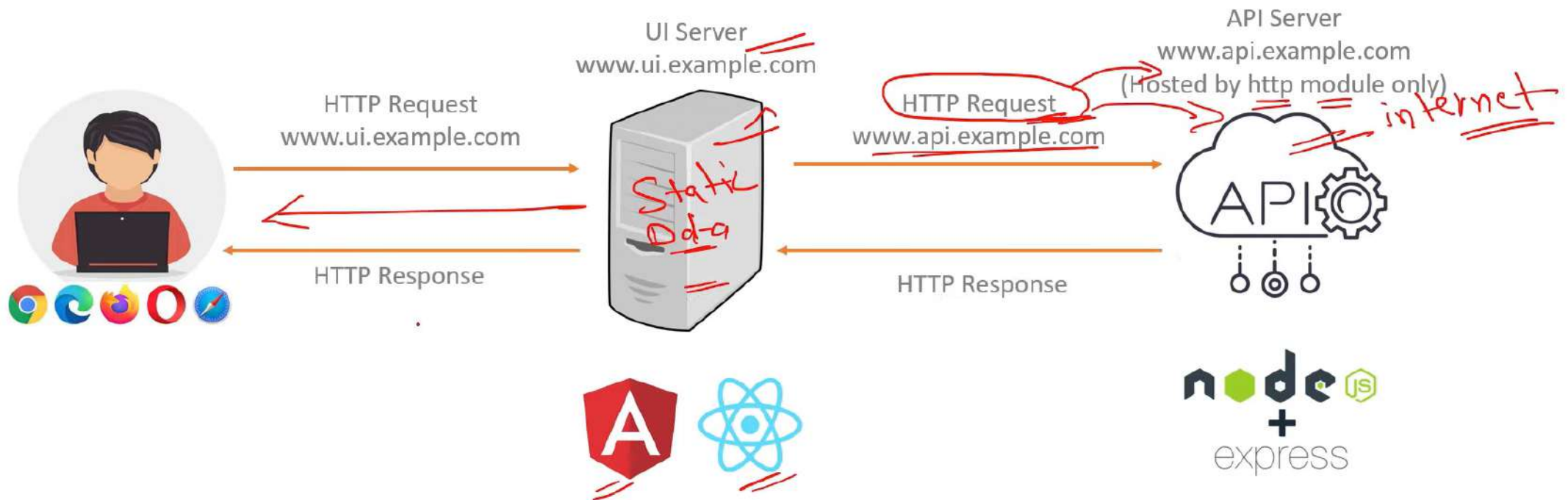
Action



Q. What is the role of http module in node? **V. IMP.**



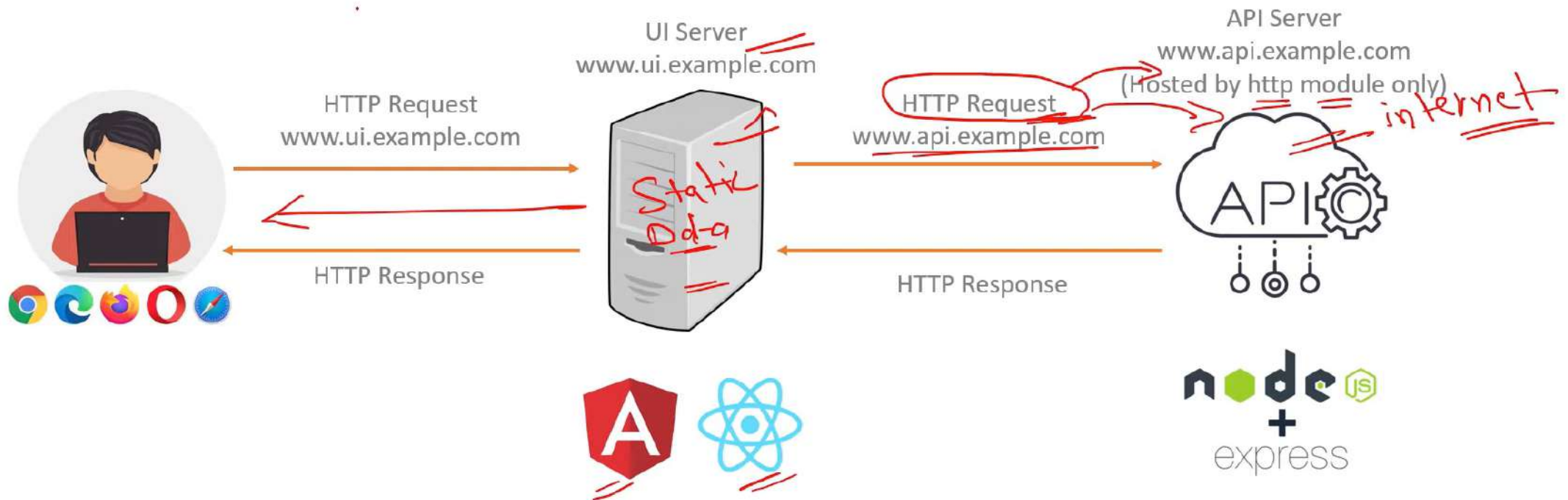
Q. What is the role of http module in node? **V. IMP.**



Q. What is the role of http module in node? **V. IMP.**



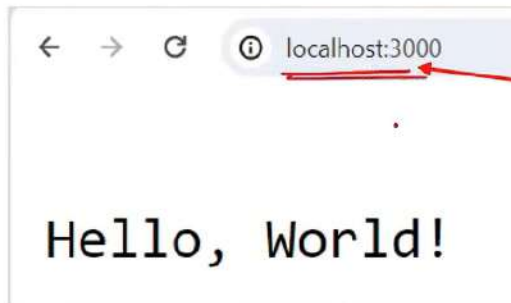
- ❖ The HTTP module can create an HTTP server that listens to server ports and gives a response back to the client.





Q. What is the role createServer() method of http module?

- ❖ The `createServer()` method of the http module in Node.js is used to **create an HTTP server**.



```
// createServer.js
// 1. Import the 'http' module to create an HTTP server
const http = require("http");

// 2. Create an HTTP server using the 'createServer' method
const server = http.createServer(req, res) => {
  // Handle incoming HTTP requests here
  res.end("Hello, World!");
};

// 3. Set port on which server will listen incoming requests
const port = 3000;

// 4. Start the server and listen on the specified port
server.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});

// 5. Run command in terminal: node createServer.js
```





Chapters

FUNDAMENTALS



1. Node.js Basics
2. Main Features of Node.js
3. Project setup & Modules
4. Top built in modules

EXPRESS

5. Express Basics
6. Middleware
7. Types of Middleware's
8. Routing - I
9. Routing - II
10. Template Engines

REST API

11. REST API Basics
12. HTTP Methods & Status Codes
13. CORS, Serialization, Deserialization, Others
14. Authentication & Authorization

NODE - OTHERS

- 15 Error Handling & Debugging
16. Security, Performance & Deployment
17. Testing & Web Socket

MONGO-DB

18. MongoDB Basics
19. Setup & CRUD operations
20. Query Operators & Projection
21. Indexes
22. Mongoose



- 23 Application based questions.
24. Scenario based short questions
25. JavaScript Essentials for Node



5. Express - Basics



Q. What are the **advantages** of using Express.js with Node.js?

Q. How do you **install Express.js** in a Node.js project?

Q. How to **create an HTTP Sever** using Express.js?

Q. How do you **create** and **start** Express.js application?



Q. What are the advantages of using Express.js with Node.js? **V. IMP.**



4

Built-in

1. Simplified Web Development

Express.js provides a lightweight framework that simplifies the process of building web applications in Node.js.

Managing

2. Middleware Support

Easy integration of middleware functions into application's request-response cycle.

→ Redirect

3. Flexible Routing System

Defining routes for handling different HTTP methods (GET, POST, PUT, DELETE, etc.) and URL patterns is easy.

HTML = Server

4. Template Engine Integration

Express.js supports various template engines making it easy to generate dynamic HTML content on the server side.



Q. How to create an HTTP Server using Express.js? **V. IMP.**



❖ Creating an HTTP Server using Express.js:

```
// Creating an HTTP server using Express.js
// Step1: Import Express
const express = require("express");

// Step 2: Create an Express application
const app = express(); // Server created

// Step 3: Define the port number
const PORT = 3000;

// Step 4: Start the Express server and
// listen on the specified port
app.listen(PORT, () => {
  console.log(`Express server running ${PORT}`);
});
```

❖ Creating an HTTP Server using Node.js http module:

```
// Creating an HTTP server using Node.js http module:
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello World\n");
});

const port = 3000;

server.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});
```



SUBSCRIBE



Q. How do you create and start Express.js application?

- ❖ Create an Express.js application by requiring the express module and calling the express() function.

```
// Create an Express.js application
const express = require("express");
const app = express();
```

- ❖ Start an Express.js server by calling the listen() method on the application object (app) and specifying the port number

```
// Start an Express.js server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on :${PORT}`);
});
```



SUBSCRIBE

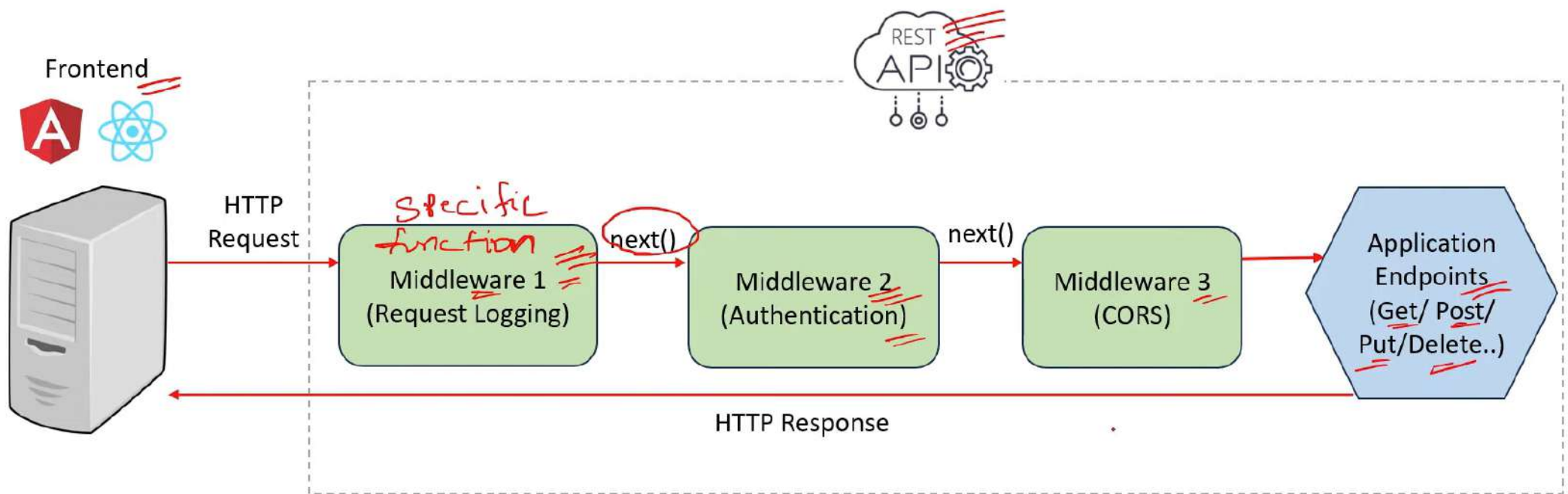
6. Express - Middleware



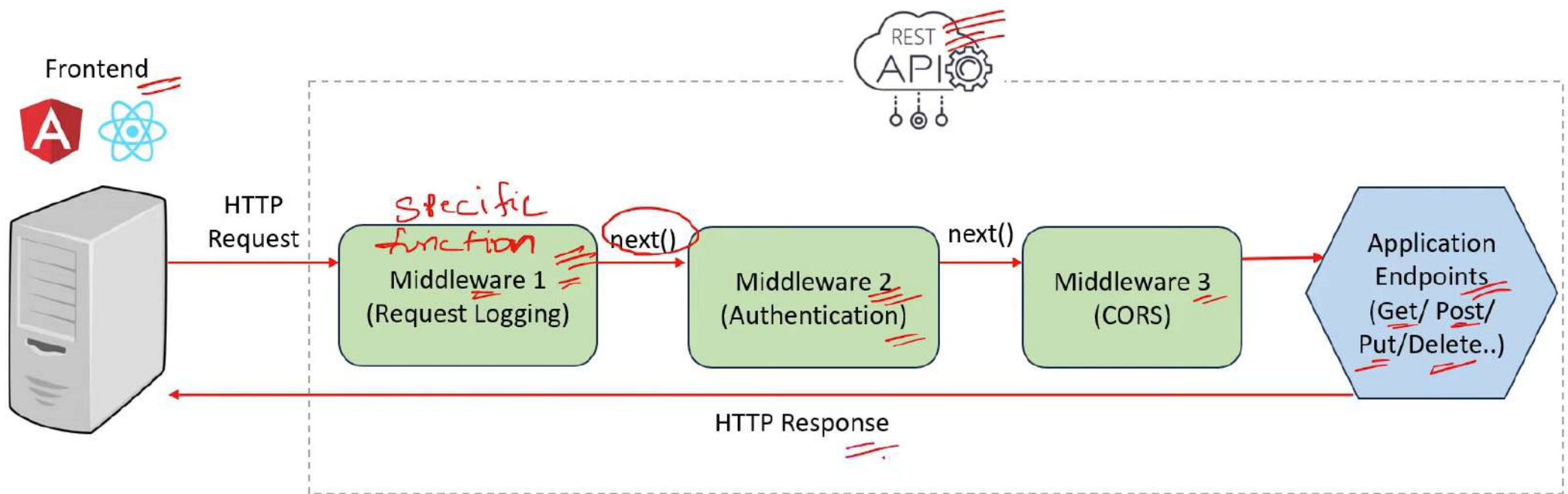
- Q. What is **Middleware** in Express.js and when to use them?
- Q. How do you **implement middleware** in Express.js?
- Q. What is the purpose of the **app.use()** function in Express.js?
- Q. What is the purpose of the **next parameter** in Express.js?
- Q. How to use middleware globally for a **specific route**?
- Q. What is **Request Pipeline** in Express?



Q. What is Middleware in Express.js and when to use them? V. IMP.



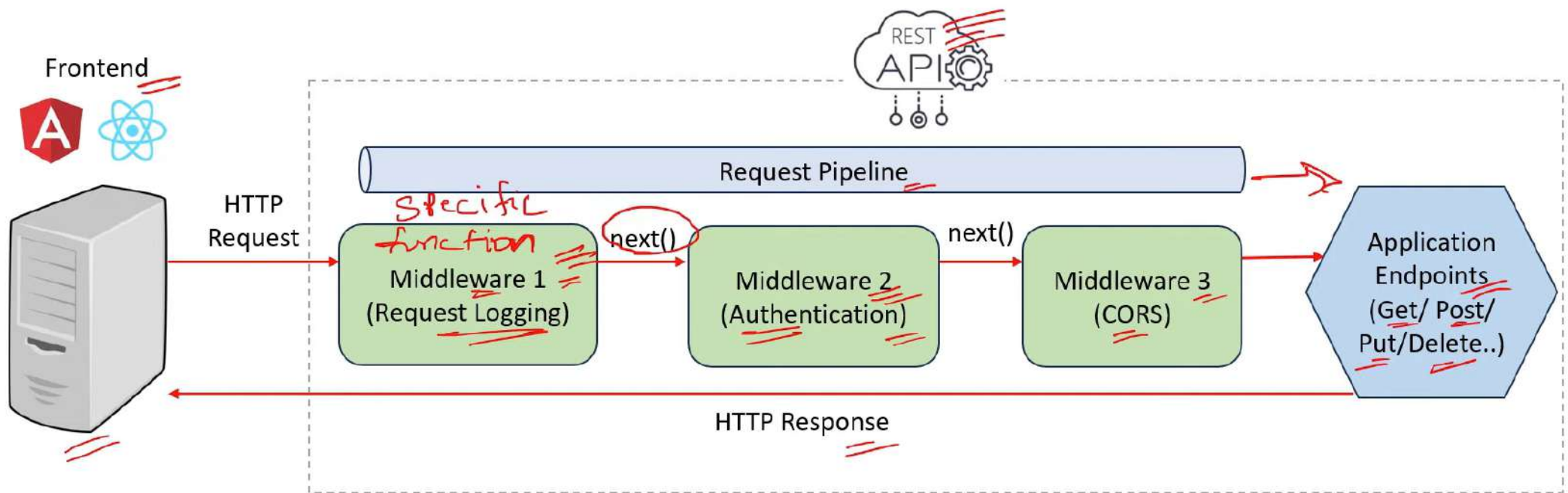
Q. What is Middleware in Express.js and when to use them? V. IMP.



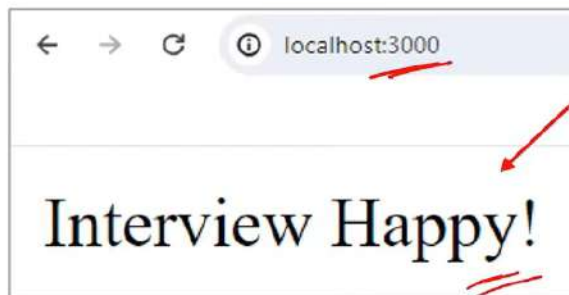
Q. What is Middleware in Express.js and when to use them? **V. IMP.**



- ❖ A middleware in Express.js is a function that handles HTTP requests, performs operations, and passes control to the next middleware.



Q. How do you implement middleware in Express.js? **V. IMP.**



```
// Create and execute middleware
const express = require("express");
const app = express();

// Define middleware function ✓
const myMiddleware = (req, res, next) => {
  // Middleware logic goes here
  res.send("Interview Happy!");
  next(); // Call the next middleware function
};

// Use middleware globally for all routes
app.use(myMiddleware);

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running ${PORT}`);
});
```

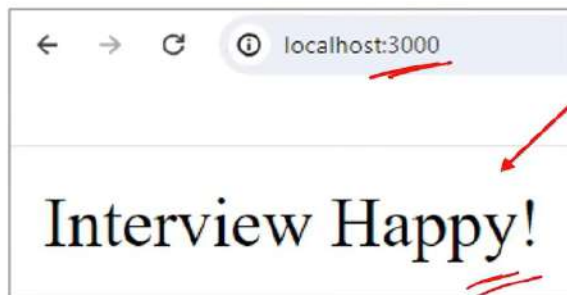




Q. How do you implement middleware in Express.js? **V. IMP.**

❖ 4 steps to use middleware:

1. Initialize an Express application.
2. Define a middleware function myMiddleware.
3. Use app.use() to mount myMiddleware globally, meaning it will be executed for every incoming request to the application.
4. Finally, we start the server by listening on a specified port (defaulting to port 3000) using app.listen().



```
// Create and execute middleware
const express = require("express");
const app = express();

// Define middleware function ✓
const myMiddleware = (req, res, next) => {
  // Middleware logic goes here
  res.send("Interview Happy!");
  next(); // Call the next middleware function
};

// Use middleware globally for all routes
app.use(myMiddleware);

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running ${PORT}`);
});
```





Q. What is the purpose of the app.use() function in Express.js?

- ❖ The app.use() method is used to **execute(mount) middleware** functions globally.

```
// Create and execute middleware

const express = require("express");
const app = express();

// Define middleware function
const myMiddleware = (req, res, next) => {
  // Middleware logic goes here
  res.send("Interview Happy!");
  next(); // Call the next middleware function
};

// Use middleware globally for all routes
app.use(myMiddleware);

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running ${PORT}`);
});
```





Q. What is the purpose of the next parameter in Express.js?

- ❖ The next parameter is a callback function which is used to **pass control to the next middleware** function in the stack.

// Output: Interview Happy

```
const express = require("express");
const app = express();

const myMiddleware1 = (req, res, next) => {
  console.log("Interview");
  next(); // Call the next middleware function
};

const myMiddleware2 = (req, res, next) => {
  console.log("Happy");
  next();
};

app.use(myMiddleware1);
app.use(myMiddleware2); //only execute if above have next method

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running ${PORT}`);
});
```





Q. How to use middleware globally for a specific route?

- ❖ Use `app.use('/specificRoute', myMiddleware)` to use middleware globally for a specific route in Express.js



```
const express = require('express');
const app = express();

const middleware = (req, res, next) => {
  res.send('Middleware for specific route');
  next();
};

// Use middleware globally for specific routes
app.use('/example', middleware);

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running ${PORT}`);
});
```

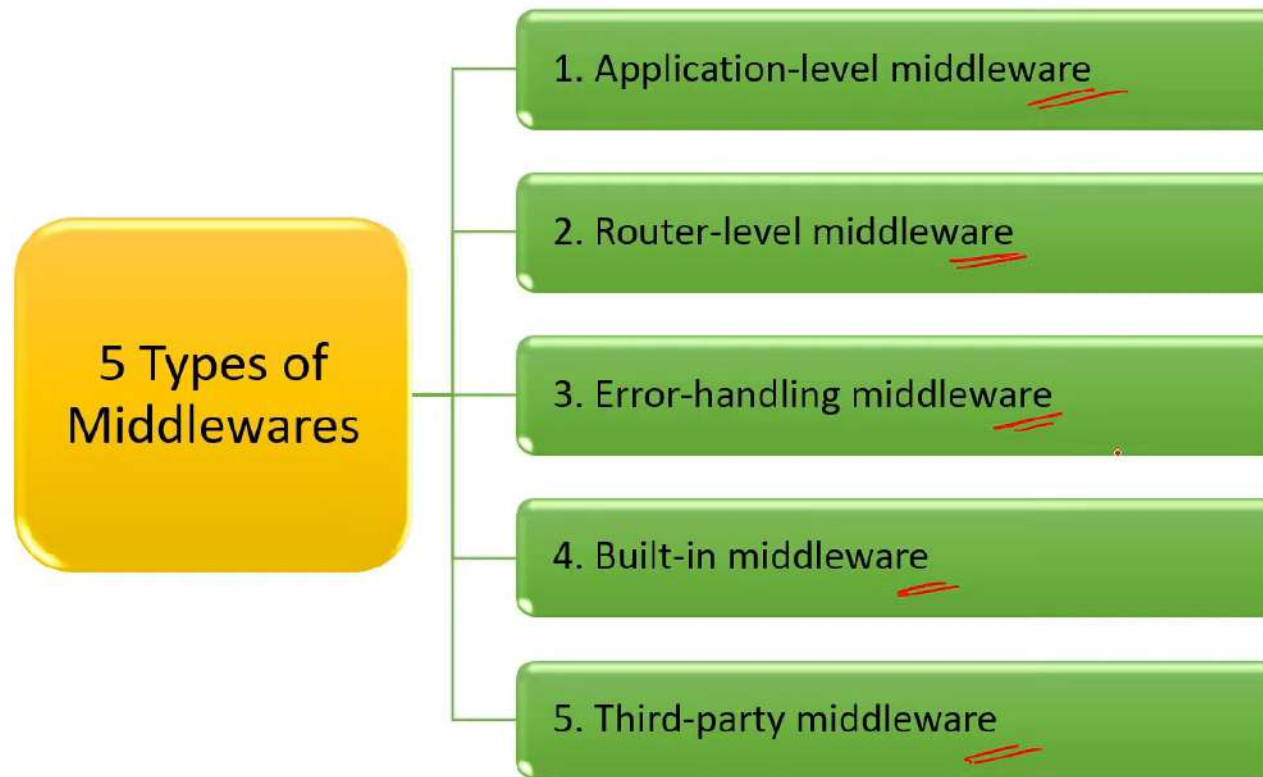

7. Express - Types of Middleware's



- Q. What are the **type of middleware's** in Express.js?
- Q. What is the difference btw **application-level** & **route-level middleware**?
- Q. What is **error handling middleware** and how to implement it?
- Q. If you have 5 middlewares then in which middleware you will do the error handling?
- Q. What is **built in** middleware? How to serve **static files** from Express.js?
- Q. What are **third-party middleware's**? Give some examples?
- Q. Can you **summarize** all the type of middleware's?
- Q. What are the **advantages** of using middleware in Express.js?



Q. What are the Types of middleware's in Express.js? **V. IMP.**



Q. What is the difference btw application-level & route-level middleware?



```
const express = require("express");
const app = express();

// Define middleware function
const myMiddleware = (req, res, next) => {
  // Middleware logic goes here
  res.send("Hello World!");
  next(); // Call the next middleware function
};

// Use middleware globally for all routes
app.use(myMiddleware);

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running ${PORT}`);
});
```

```
const express = require('express');
const app = express();

const middleware = (req, res, next) => {
  res.send('Middleware for specific route');
  next();
};

// Use middleware globally for specific routes
app.use('/example', middleware);

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running ${PORT}`);
});
```





Q. What is error handling middleware and how to implement it?

- ❖ Error handling middleware in Express is a special kind of middleware used to **manage errors** happening while handling incoming requests.



```
const express = require("express");
const app = express();

// Middleware generating error
app.use((req, res, next) => {
  // Simulate an error
  next(new Error("An error occurred"));
});

// Error-handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send("Something went wrong!");
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on ${PORT}`);
});
```

Error: An error occurred
at D:\Node\Node-Code\Express-Middleware\error.js:7:8
at Layer.handle [as handle_request] (D:\Node\Node-Code\



Q. What is error handling middleware and how to implement it?

- ❖ Error handling middleware in Express is a special kind of middleware used to **manage errors** happening while handling incoming requests.
- ❖ To implement error handling in Express, define middleware with four parameters (err, req, res, next). Here the additional **error object parameter** will be used for error handling.



```
const express = require("express");
const app = express();

// Middleware generating error
app.use((req, res, next) => {
  // Simulate an error
  next(new Error("An error occurred"));
});

// Error-handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send("Something went wrong!");
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on ${PORT}`);
});
```

Error: An error occurred
at D:\Node\Node-Code\Express-Middleware\error.js:7:8
at Layer.handle [as handle_request] (D:\Node\Node-Code\

Q. If you have 5 middlewares then in which middleware you will do the error handling? 

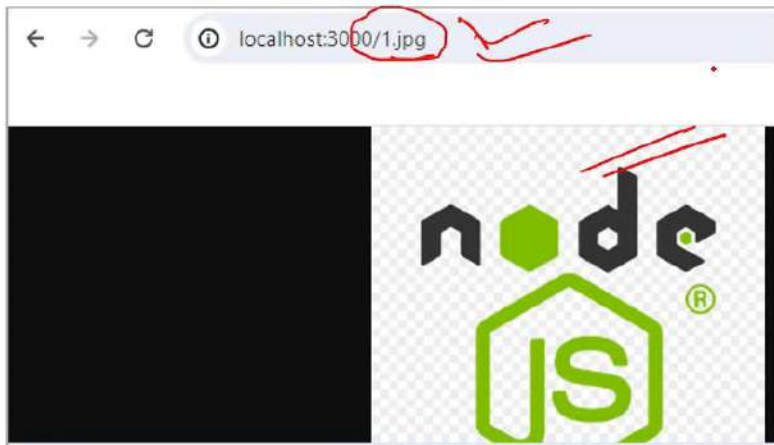
- ❖ In case of multiple middleware, error-handling middleware should be **defined at last** (after all other middleware's) because when an error occurs, Express.js will search for the next error-handling middleware skipping any regular middleware or route handlers.

```
// Multiple middlewares
app.use(middleware1);
app.use(middleware2); // Error occurred
app.use(middleware3); X
app.use(middleware4); X

// Error-handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send("Something went wrong!");
});
```



Q. What is built in middleware? How to serve static files from Express.js?



```
const express = require("express");
const app = express();

// Serve static files from the 'public' directory
app.use(express.static("public"));

// Other routes and middleware can be defined here

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on :${PORT}`);
});
```





Q. What are third-party middleware's? Give some examples?

- ❖ Third-party middleware in Express.js are modules developed by third-party developers (not part of the core Express.js).
- ❖ Third-party middleware's provide functionalities like logging, security, body parsing, and compression.
- ❖ Example of third-party middleware's: morgan, helmet, body-parser, compression etc.

Community →

```
// Third-party middlewares
// npm install helmet body-parser compression

const express = require('express');
const helmet = require('helmet');
const bodyParser = require('body-parser');
const compression = require('compression');

const app = express();

// Use the helmet middleware for setting HTTP security headers
app.use(helmet());

// Use the body-parser middleware for parsing request bodies
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));

// Use the compression middleware for compressing HTTP responses
app.use(compression()); Performance
```

Q. Can you summarize all the type of middleware's?



1. Application-level middleware

Middleware applied to all routes; commonly used for logging, authentication, etc.

2. Router-level middleware

Middleware specific to certain routes; applied using `router.use()` or `router.route().use()`.

3. Built-in middleware

Pre-packaged middleware included with Express.js, like for serving static files.

4. Error-handling middleware

Middleware for handling errors; declared after other middleware, triggered on errors.

5. Third-party middleware

Middleware developed by others, not part of Express.js core; adds various functionalities.



Q. What are the advantages of using middleware in Express.js? **V. IMP.**



1. Modularity

*small
functions*

Middleware allows you to modularize your application's functionality into **smaller, self-contained units**. Each middleware function can handle a specific task or concern, such as logging, authentication, or error handling.

2. Reusability

maintain

Middlewares can be reused at **multiple places** and that makes application code easier to maintain.

3. Improved Request Handling

Middleware functions have access to both the request (req) and response (res) objects which enables you to perform **validations on request** or modify the response before sending it back to the client.

4. Flexible Control Flow

Route

Middleware functions can be applied globally to all routes or selectively to specific routes, allowing you to control the flow of request processing in your application.

5. Third-party Middleware's

less code

Express.js offers a wide range of third-party middleware packages that provide additional functionality. For eg: body-parser, cors etc.

8. Express - Routing - I

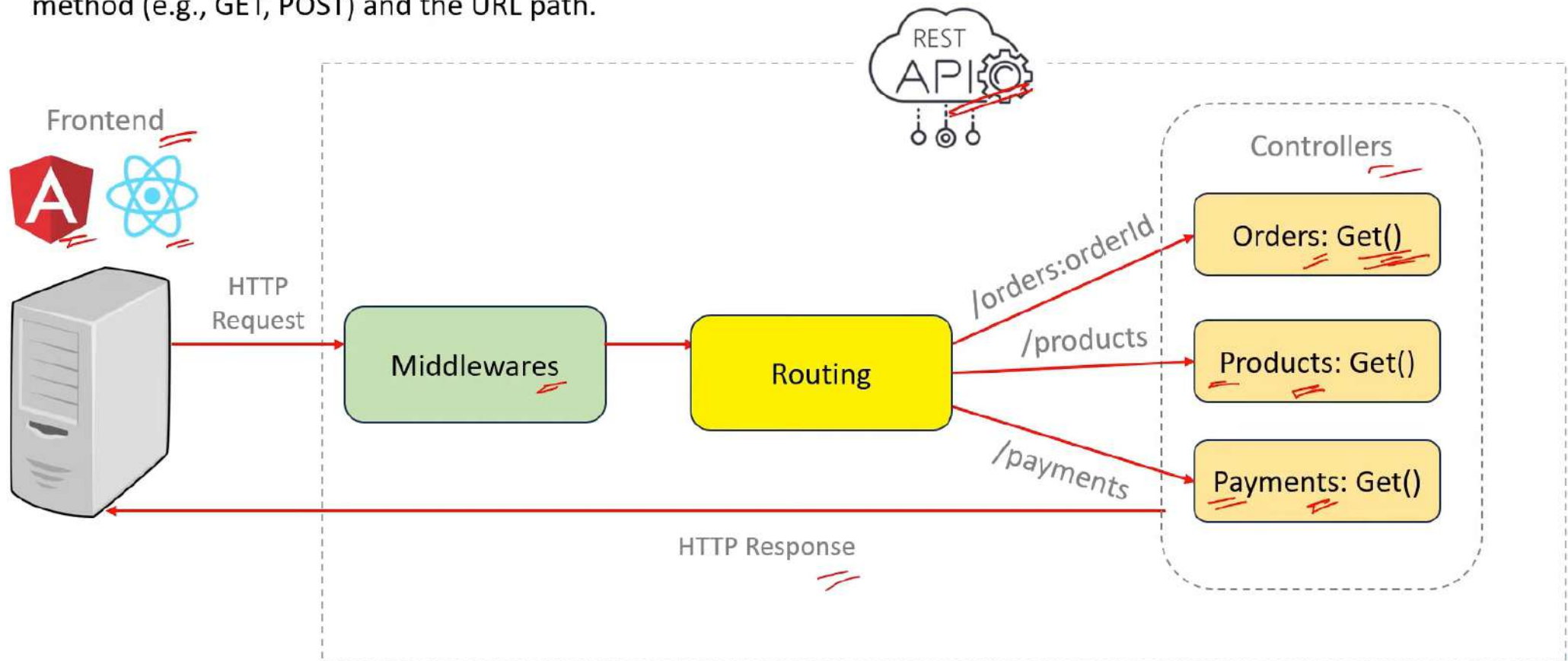


- Q. What is **Routing** in Express.js?
- Q. What is the difference between **middleware** & **routing** in Express?
- Q. How to **implement** routing? How do you **define routes** in Express.js?
- Q. How to handle **Routing** in Express.js **real applications**?
- Q. What are **Route Handlers**?
- Q. What are **Route Parameters** in Express.js?



Q. What is Routing in Express.js? **V. IMP.**

- ❖ Routing is the process of directing incoming HTTP requests to the appropriate handler functions based on the request's method (e.g., GET, POST) and the URL path.



Q. What is the difference between middleware & routing in Express?



Middleware

1. Middleware are functions.

2. Middleware functions can access and the request and response objects, then they can:

- a) perform some actions(logic like authorization)
- b) End the request-response cycle
- c) Call the next middleware function.

Routing

1. Routing is a process.

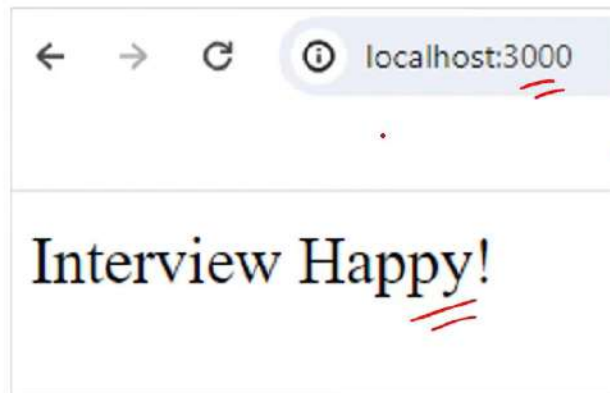
Routing is the process of directing incoming HTTP requests to the appropriate handler functions(Get, Put, Post/ Delete).





Q. How to implement routing? How do you define routes in Express.js?

- ❖ To implement routing first define the routes.
- ❖ In Express.js, routes are defined using the **app.METHOD() functions**. (where METHOD is the HTTP request method (e.g., GET, POST, PUT, DELETE) and app is an instance of the Express application).



```
const express = require("express");
const app = express();

// Define a route for handling GET requests
app.get("/", (req, res) => {
  res.send("Interview Happy!");
});

// Define a route for handling POST requests
app.post("/login", (req, res) => {
  // Handle login logic
});

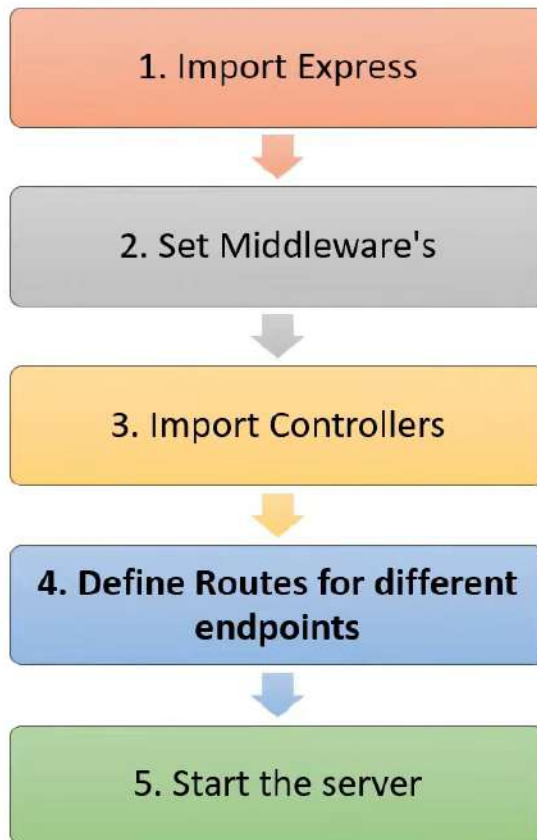
// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on ${PORT}`);
});
```



Q. How to handle Routing in Express.js real applications?



❖ 5 steps for setup routing:



```
const express = require("express");
const app = express();

// Middlewares

// Import Controllers
const ordersController = require("./controllers/ordersController");
const productsController = require("./controllers/productsController");
const paymentsController = require("./controllers/paymentsController");

// Routes
app.get("/orders/:orderId", ordersController.getOrderById);
app.get("/products", productsController.listProducts);
app.get("/payments", paymentsController.paymentInfo);

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```





- ```
const express = require("express");
const app = express();

// Define a route for handling GET requests
app.get("/", (req, res) => {
 res.send("Interview Happy!");
});

// Define a route for handling POST requests
app.post("/login", (req, res) => {
 // Handle login logic
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
 console.log(`Server is running on ${PORT}`);
});
```

## Q. What are Route Parameters in Express.js?



- ❖ Route parameters in Express.js allow you to capture dynamic values from the URL paths.
- ❖ Route parameters can be accessed by using **req.params** object.

```
const express = require("express");
const app = express();

// Define a route with a route parameter
app.get("/users/:userId", (req, res) => {
 // Access the value of the userId parameter
 const userId = req.params.userId;
 res.send(`User ID: ${userId}`);
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
 console.log(`Server is running on :${PORT}`);
});
```





# 9. Express - Routing - II

---



- Q. What are **Router object** & **Router Methods** and how to implement them?
- Q. What are the types of **Router Methods**?
- Q. What is the difference between **app.get()** and **router.get()** method?
- Q. What is **express.Router()** in Express.js?
- Q. Share a real application use of Routing?
- Q. What is **Route Chaining** in Express.js?
- Q. What is **Route Nesting** in Express.js?
- Q. How to implement **route nesting** in Express.js?



## Q. What are Router object & Router Methods and how to implement them? **V. IMP.**



- ❖ The router object is a mini version of an Express application which is used for handling routes.
- ❖ Router Methods are functions provided by the Router object to define routes for different HTTP methods (GET, POST, DELETE, etc.).

### ❖ Implement router method

```
// router.js
const express = require("express");
// Create a router object
const router = express.Router();

// Define a route for the root URL ('/')
router.get("/", (req, res) => {
 res.send("Interview Happy");
});

// Export the router object
module.exports = router;
```

### ❖ Use router method

```
// app.js
const express = require("express");
const router = require("./router");
const app = express();

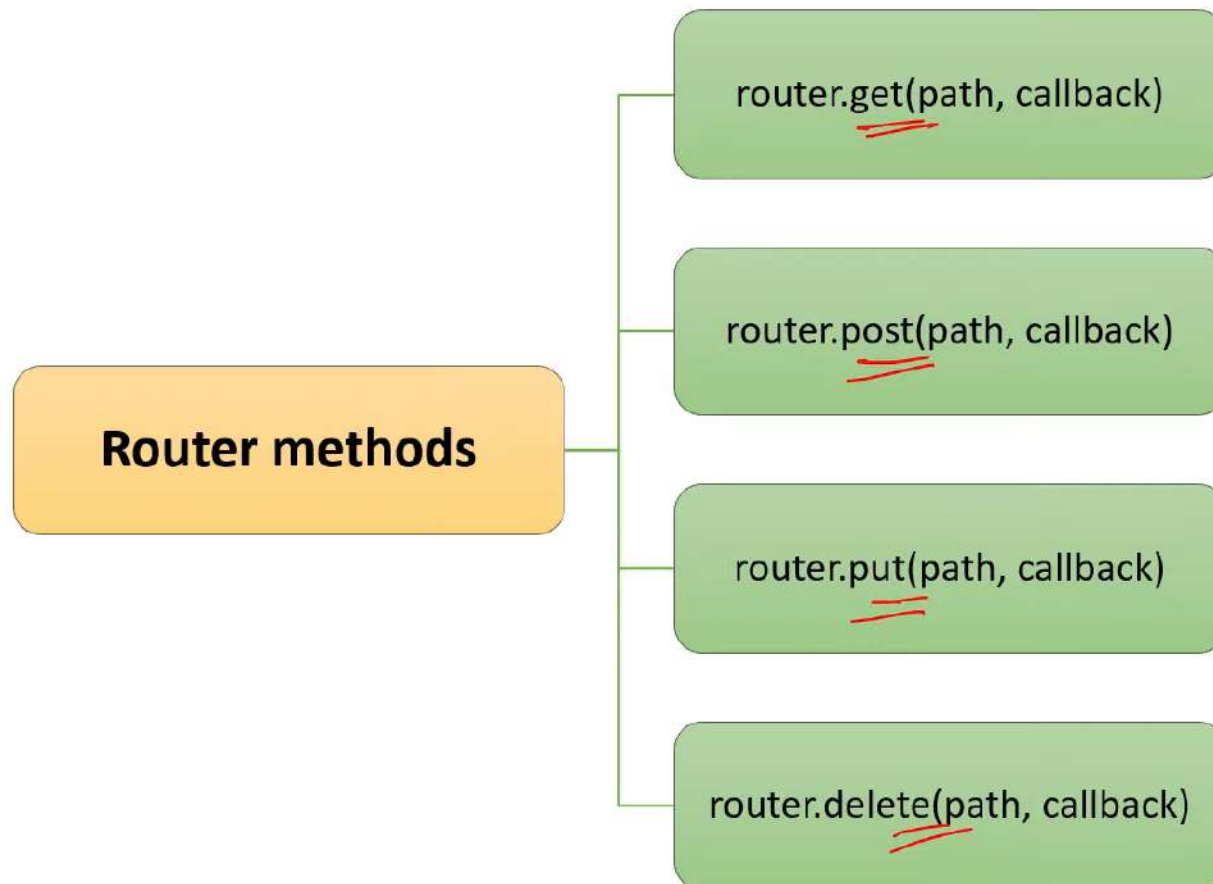
// Mount the router object on a path
app.use("/api", router);

// Start the server
app.listen(3000, () => {
 console.log("Server is running");
});
```



SUBSCRIBE

Q. What are the types of Router Methods?



Q. What is the difference between app.get() and router.get() method?



```
// app-get.js
const express = require("express");
const app = express();

app.get("/", (req, res) => {
 res.send("Interview Happy");
});

app.listen(3000, () => {
 console.log("Server is running");
});
```

```
// router.js
const express = require("express");
const router = express.Router();

// Define a route for the root URL ('/')
router.get("/", (req, res) => {
 res.send("Interview Happy");
});

// Export the router object
module.exports = router;
```



1:56:30 / 3:29:12



SUBSCRIBE



## Q. What is the difference between app.get() and router.get() method?



1. The app.get() method is used to define routes **directly on the application object**.
2. Routes defined using app.get() are automatically mounted on the root path (/).
3. Routes defined using app.get() are not modular and cannot be reused in other applications.

```
// app-get.js
const express = require("express");
const app = express();

app.get("/", (req, res) => {
 res.send("Interview Happy");
});

app.listen(3000, () => {
 console.log("Server is running");
});
```

~~Reuse~~

1. The router.get() method is used to define routes on a router object.
2. Routes defined using router.get() are not automatically mounted; they must be explicitly mounted **using app.use()**.
3. Routes defined using router.get() are modular and can be **reused in other applications** by exporting the router object.

```
// router.js
const express = require("express");
const router = express.Router();

// Define a route for the root URL ('/')
router.get("/", (req, res) => {
 res.send("Interview Happy");
});

// Export the router object
module.exports = router;
```

Reuse

app.use



Q. What is express.Router() in Express.js?



- ❖ express.Router is a class in Express.js that returns a new router object.

```
// router-get.js
const express = require("express");
const router = express.Router();

// Define a route for the root URL ('/')
router.get("/", (req, res) => {
 | res.send("Interview Happy");
});

// Export the router object
module.exports = router;
```



SUBSCRIBE

Q. Share a real application use of Routing?



- ❖ Routing is used for authenticating requests based on the token available in request header.

```
const express = require('express');
const app = express();
const router = express.Router();

// Route-level middleware for authentication
const authenticate = (req, res, next) => {
 if (req.headers.authorization === 'Bearer myToken') {
 next(); // Proceed to the next middleware
 } else {
 res.status(401).send('Unauthorized');
 }
};

// Apply route middleware to specific route
router.get('/protected', authenticate, (req, res) => {
 res.send('This is a protected route');
});

// Mount the router
app.use('/api', router);
```

